

CS-553 Midterm Exam

Take Home - Summer 2026

Schrimpsher

This is an applied take-home exam, not a pile of trivia questions. The goal is to prove you understand the stack:

socket → **HTTP** → **Express route** → **middleware** → **client** → **OpenAPI contract**

The exam excludes authentication and database work.

##

Coverage: Sockets, HTTP/JSON, REST-style resource design, Express routes, middleware, basic clients, and OpenAPI.

Not covered: databases, authentication, authorization, JWTs, OAuth, messaging, WebSockets, GraphQL, deployment.

Submission: Submit a GitHub repository link or ZIP file containing:

1. `answers.md`
2. Express server source code
3. basic client source code
4. `openapi.yaml`
5. `README.md` with setup/run/test instructions

Allowed resources: Course notes, slides, labs, official Node/Express/OpenAPI documentation.

Not allowed: Collaboration with classmates, copying from online solutions, or using AI tools to generate code or written answers. AI may be used only as a study aid for understanding concepts, consistent with the course AI policy.

Part 1 — Conceptual Foundations

20 points

Answer each question in `answers.md`. Use complete sentences, but do not write a novel.

1. Sockets vs. HTTP

Explain the difference between a raw TCP socket server and an HTTP server. Your answer should include:

- what a socket provides,
- what HTTP adds on top,
- why most web APIs do not directly expose raw socket protocols.

5 points

2. Request/Response

Describe the request/response pattern. Then explain how it appears in:

- a TCP command server,
- an HTTP API,
- an Express route handler.

5 points

3. Statelessness

Explain what it means for an API to be stateless. Give one advantage and one disadvantage of stateless design.

5 points

4. HTTP Status Codes

For each situation, choose an appropriate HTTP status code and briefly justify it.

Situation	Status Code
A new resource was successfully created	
The client requested an item that does not exist	
The client sent JSON missing a required field	
The server had an unexpected error	
A successful request returns JSON data	

5 points

Part 2 — API Design

15 points

You are designing an API for a small **Course Task Tracker**. The API manages tasks for a class.

Each task has:

```
{
  "id": "1",
  "title": "Watch Week 3 lecture",
  "course": "CS453",
  "completed": false
}
```

Design a REST-style API for this resource.

In `answers.md`, provide:

1. Resource URIs

List URIs for:

- getting all tasks,
- getting one task by id,
- creating a task,
- replacing a task,
- partially updating a task,
- deleting a task.

6 points

2. Method Semantics

For each route, identify the HTTP method and explain whether it is:

- safe,
- idempotent,
- neither.

6 points

3. JSON Representation

Provide one valid JSON example for creating a new task.

3 points

Part 3 — Express API Implementation

30 points

Implement an Express API for the Course Task Tracker.

Use an in-memory array. Do **not** use a database.

Your server must support these routes:

Method	Route	Behavior
GET	<code>/health</code>	returns a simple health message
GET	<code>/api/tasks</code>	returns all tasks
GET	<code>/api/tasks/:id</code>	returns one task or <code>404</code>

Method	Route	Behavior
POST	/api/tasks	creates a new task
PUT	/api/tasks/:id	replaces an existing task
PATCH	/api/tasks/:id	updates part of an existing task
DELETE	/api/tasks/:id	deletes a task

Requirements:

- Use JSON request and response bodies.
- Generate or assign ids.
- Return appropriate status codes.
- Validate required fields for `POST`.
- Return `400` for bad input.
- Return `404` for missing tasks.
- Use `204 No Content` or a reasonable JSON response for successful deletes.
- Keep the code readable and organized.

Suggested structure:

```
src/  
  server.js  
  routes/  
    tasks.js  
  middleware/  
    logger.js  
    errorHandler.js
```

30 points

Part 4 — Middleware

10 points

Implement at least two pieces of middleware.

Required:

1. A request logger that records:
 - HTTP method,
 - path,
 - response status,
 - time taken.
2. A validation middleware for creating or updating tasks.

In `answers.md`, briefly explain why these are middleware concerns instead of being repeated manually inside every route.

10 points

Part 5 — Basic Client

10 points

Write a small client program that calls your API.

This can be a Node script using `fetch`.

The client should demonstrate:

1. calling `/health`,
2. creating a task,
3. listing all tasks,
4. getting one task by id,
5. updating a task,
6. deleting a task.

The client does not need a GUI. Console output is fine.

Example:

```
node client.js
```

10 points

Part 6 — OpenAPI Specification

15 points

Create an `openapi.yaml` file documenting your API.

It must include:

- OpenAPI version,
- API title and version,
- server URL,
- paths for all required routes,
- path parameter for `:id`,
- request body schemas for `POST`, `PUT`, and `PATCH`,
- response schemas,
- common error responses such as `400` and `404`,
- a reusable `Task` schema under `components.schemas`.

15 points

Part 7 — Reflection

10 points

In `answers.md`, answer the following:

1. Code vs. Contract

Explain the difference between an Express route implementation and an OpenAPI specification.

4 points

2. Drift

Give two examples of how code and OpenAPI documentation can drift apart.

3 points

3. Client Impact

Explain why inaccurate API documentation can cause problems for client developers.

3 points

Part 8 Graduate

10 additional points

Choose one:

Option A — Architecture Critique

Suppose this task API grows into a larger system with users, assignments, grades, and notifications.

Explain which parts should remain in one service and which might eventually become separate services.

Defend your answer using client/server, middleware, and service-boundary reasoning.

Option B — Contract-First Design

Explain the advantages and disadvantages of designing the OpenAPI specification before writing the Express code.

Your answer should discuss team communication, testing, client development, and the risk of implementation drift.